

Code Lifecycle — From Entry Point to Output

This document traces one thing only: **the exact sequence of files and function calls a request passes through**, from the moment it hits the server to the moment a response goes back out. Each step names one file, its responsibility, and what it hands off to next. It complements [LEARNING.md](#) (which explains *why* the system is built this way) by focusing purely on the *call chain*.

There are two ways in, and one offline step that has to happen before either of them can produce real advice:

- **Step 0 (offline, one-time):** build the knowledge base the advice is grounded in.
- **Entry Point A:** a direct REST call — `POST /analyze-report`.
- **Entry Point B:** an AI assistant (ChatGPT) calling the `analyze_blood_report` MCP tool.

Both entry points converge on the exact same orchestrator function, `pipeline.py → analyze_report()`, so everything from Step 2 onward is identical no matter which door the request came through.

Step 0 — Offline knowledge-base build (prerequisite, not part of a live request)

Run once manually (`python3 build_vector_store.py`), and again any time an article changes. Nothing in the live server calls this automatically.

File	Responsibility
load_articles.py	Reads every <code>.md</code> file in <code>articles/</code> into <code>{filename: text}</code> .
chunker.py	<code>chunk_text()</code> splits each article into overlapping word chunks (60 words, 15-word overlap) so no sentence gets cut across a chunk boundary without appearing intact somewhere.
embedder.py	<code>embed_text()</code> calls OpenRouter's embedding model (<code>nvidia/llama-nemotron-embed-v1-1b-v2:free</code>) and turns one chunk of text into a vector of numbers.
build_vector_store.py	Chains the three files above: <code>load_articles()</code> → <code>chunk_text()</code> per article → <code>embed_text()</code> per chunk → writes every <code>{article, chunk_index, text, embedding}</code> record to <code>vector_store.json</code> .

Output of this step: `vector_store.json` — the flat-file "database" that Step 5 below searches at request time.

Step 1 (Entry Point A) — `main.py`

- **Responsibilities:** the FastAPI app. Listens on port 8001 for HTTP requests. Validates that an upload is actually an image, copies it to a temp file, and hands that file path to the orchestrator. Also mounts the MCP server (Entry Point B) and the static file server (`/uploads`) into the same process.
- **Triggering route:** `POST /analyze-report`
- **Key function:** `analyze_report_endpoint(file: UploadFile)`
- **Calls into:** `pipeline.py`

```
result = analyze_report(tmp_path)    # main.py -> pipeline.py
return result                        # pipeline.py's dict, serialized straight to JSON
```

Step 1 (Entry Point B) — `mcp_server/server.py`

- **Responsibilities:** exposes one MCP tool, `analyze_blood_report(image_url)`, that an AI assistant can call. Fetches the image bytes itself (`httpx.get`) from wherever `image_url` points — usually this same app's own `/uploads/...` static route — writes them to a temp file, and hands that file path to the same orchestrator as Entry Point A.
- **Key function:** `analyze_blood_report(image_url: str) -> dict`
- **Calls into:** `pipeline.py` (identical call to Entry Point A — `analyze_report(tmp_path)`)
- **Rejoins the main chain at:** Step 2 below. Everything from here on is the same code path regardless of which entry point was used.

Step 2 (Orchestrator + Stage 1) — `pipeline.py`

- **Responsibilities:** the one function both entry points call — `analyze_report(image_path) -> dict`. It owns the top-level sequence (Stage 1 → 2 → 3+4) and directly contains Stage 1 itself.
- **Key function:** `analyze_report()`, plus `extract_biomarkers()` (Stage 1, vision extraction)
- **What Stage 1 does:** base64-encodes the image, sends it in a multimodal chat message to a vision model on OpenRouter (`google/gemma-4-31b-it:free` , with two fallback models), and parses the model's JSON reply into `{biomarker_name: value}`.
- **Calls into:** OpenRouter (AI call) → then `categorize.py`

```
biomarkers = extract_biomarkers(image_path)    # Stage 1 – pipeline.py calls OpenRouter
categorized = categorize(biomarkers)           # Stage 2 – pipeline.py -> categorize.py
...
advice = generate_advice(categorized, vector_store) # Stage 3+4 -> advisor.py
```

Step 3 (Stage 2) — `categorize.py` + `reference_ranges.py`

- **Responsibilities:** pure comparison logic, no AI call. For every extracted biomarker, looks up its `{min, max, unit}` in `reference_ranges.py` and tags it `Low` / `High` / `Normal` / `Unknown` (no reference range).
- **Key function:** `categorize(biomarkers: dict) -> dict`
- **Calls into:** nothing further — returns straight back up to `pipeline.py`.

Step 4 (Stage 3+4 orchestrator) — `advisor.py`

- **Responsibilities:** turns the categorized findings into grounded, human-readable advice. Filters down to only the abnormal biomarkers (returns a canned "all normal" message immediately if there are none — no AI call at all in that case). For each abnormal biomarker, builds a search query and calls `retriever.py` to fetch the most relevant knowledge-base paragraph. Assembles all of that into one prompt and sends it to a text model on OpenRouter, with retry/fallback across models.
- **Key functions:** `generate_advice(categorized, vector_store)`, `call_model_with_fallback(prompt)`
- **Calls into:** `retriever.py` (per abnormal biomarker), then OpenRouter directly for the final generation call.

Step 5 (Stage 3 engine) — `retriever.py`

- **Responsibilities:** the search engine `advisor.py` calls into. Loads `vector_store.json` (also used once up-front by `pipeline.py` before calling `advisor.py`), embeds the incoming query via `embedder.py`, computes cosine similarity between the query vector and every stored chunk's vector, and returns the top matches.
- **Key functions:** `load_vector_store()`, `retrieve_relevant_chunks(query, vector_store, top_k)`, `cosine_similarity()`
- **Calls into:** `embedder.py` (to embed the query text)

Step 6 (embedding) — `embedder.py`

- **Responsibilities:** the single place that turns any text into a vector, via OpenRouter's embedding model. Used both by the offline build (Step 0) and here, at request time, so that queries and articles are embedded with the exact same model and are numerically comparable.
- **Key function:** `embed_text(text) -> list`
- **Calls into:** OpenRouter (AI call) — then returns the vector back up to `retriever.py`, which returns matching chunks back up to `advisor.py`.

Step 7 (Output) — back to `pipeline.py`, then the entry point

- `advisor.py` returns the finished advice string to `pipeline.py`.
 - `pipeline.py` assembles `{biomarkers, categorized, advice}` and returns it to whichever entry point called it.
 - **Entry Point A** (`main.py`): FastAPI serializes that dict straight to a JSON HTTP response.
 - **Entry Point B** (`mcp_server/server.py`): FastMCP serializes the same dict as the MCP tool's result, which the AI assistant turns into a conversational reply.
-

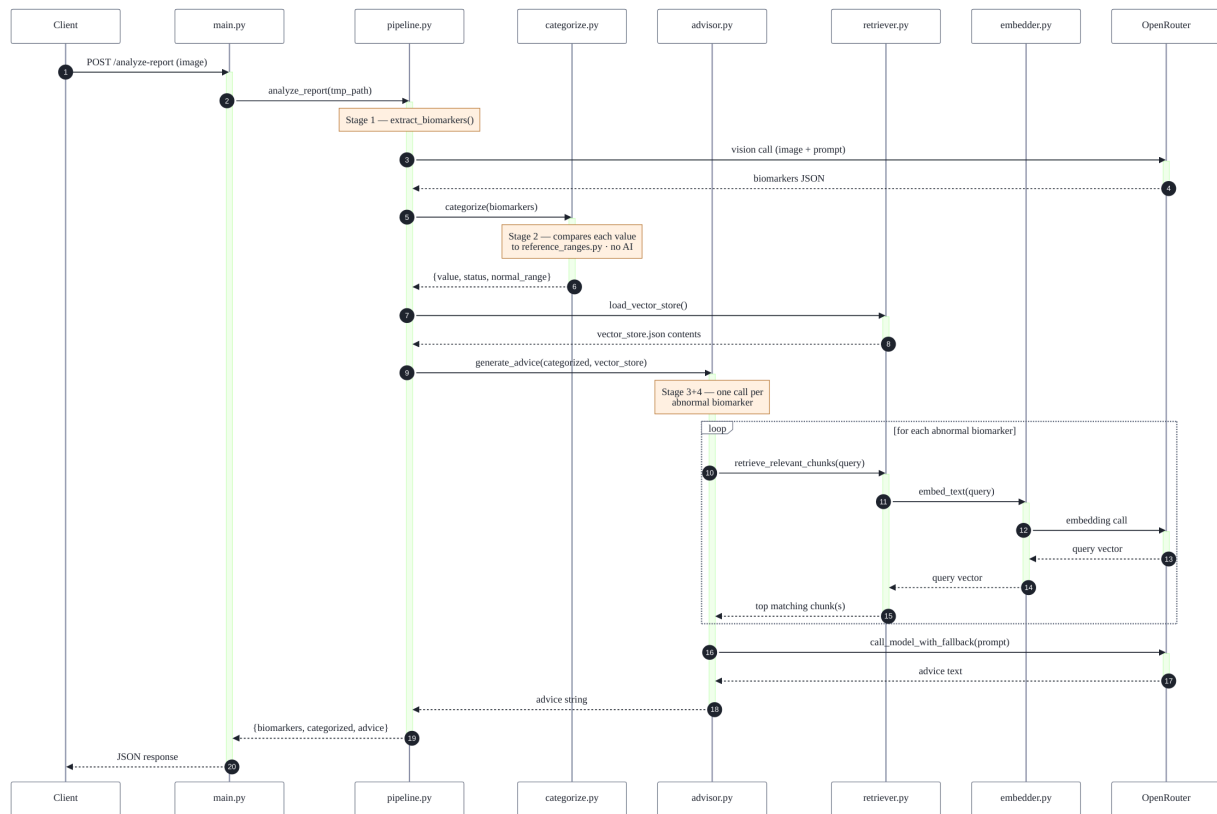
Aside — Mock mode short-circuit

When `MOCK_AI=true` in `.env`, two files are bypassed entirely so the app's structure can be exercised without spending AI quota:

- `pipeline.py` → `extract_biomarkers()` skips the vision call and returns `MOCK_BIOMARKERS` from `mock_data.py` directly.
- `pipeline.py` → `analyze_report()` skips `advisor.py` entirely and uses the fixed `MOCK_ADVICE` string from the same file.

Stage 2 (`categorize.py`) still runs for real in mock mode — only the two AI-calling stages are replaced.

Connect the Dots — the Lifecycle, File by File



► Mermaid source for the diagram above

The chain, as plain arrows

```

main.py (POST /analyze-report)
  -> pipeline.py : analyze_report(tmp_path)

      -> pipeline.py : extract_biomarkers()           [Stage 1]
          -> OpenRouter                               (vision call)

      -> categorize.py : categorize(biomarkers)       [Stage 2]
          -> reference_ranges.py                     (lookup table, no call)

      -> retriever.py : load_vector_store()
          -> vector_store.json                       (reads file)

      -> advisor.py : generate_advice(categorized, vs) [Stage 3+4]
          -> retriever.py : retrieve_relevant_chunks() (once per abnormal biomarker)
              -> embedder.py : embed_text()
                  -> OpenRouter                       (embedding call)
                      -> OpenRouter : call_model_with_fallback() (generation call)

      <- pipeline.py returns {biomarkers, categorized, advice}
      <- main.py serializes it to a JSON response
  
```

Entry Point B swaps only the first and last hop:

```

mcp_server/server.py : analyze_blood_report(image_url)
    -> httpx.get(image_url)                                (fetches the image itself)
    -> pipeline.py : analyze_report(tmp_path)                [same chain as above]
    <- returns the dict as the MCP tool result to the AI assistant

```

Quick Reference — File → Role → Neighbors

File	Role in the chain	Called by	Calls into
main.py	Entry Point A · REST routes	HTTP client	pipeline.py
mcp_server/server.py	Entry Point B · MCP tool	AI assistant (MCP client)	main.py 's static route (for the image), pipeline.py
pipeline.py	Orchestrator + Stage 1 (vision extraction)	main.py , mcp_server/server.py	OpenRouter, categorize.py , retriever.py , advisor.py
categorize.py	Stage 2 (categorization, no AI)	pipeline.py	reference_ranges.py
reference_ranges.py	Static lookup table	categorize.py	nothing
advisor.py	Stage 3+4 orchestrator (retrieval + generation)	pipeline.py	retriever.py , OpenRouter
retriever.py	Stage 3 engine (similarity search)	pipeline.py , advisor.py	embedder.py , vector_store.json
embedder.py	Text → vector conversion	retriever.py , build_vector_store.py	OpenRouter
mock_data.py	Canned fake results	pipeline.py (only when MOCK_AI=true)	nothing
load_articles.py	Reads articles/*.md	build_vector_store.py	nothing
chunker.py	Splits article text into chunks	build_vector_store.py	nothing
build_vector_store.py	Offline build script (Step 0)	run manually	load_articles.py , chunker.py , embedder.py

No file in this project calls anything not listed in its own row — this table is the complete call graph.